

Rapport technique

Adaptation de V-JEPA-AC vers un prédicteur direct multi-horizon causal

Projet : V-JEPA-AC-para1

Date du rapport : 20 juin 2026

Résumé

Ce rapport décrit les modifications apportées au modèle `ac_video_jepa`. L'objectif est de remplacer le rollout autoregressif mono-step par une prédiction directe multi-horizon, calculée en un seul forward pass, tout en garantissant une causalité stricte sur la séquence d'actions.

Table des matières

1	Contexte et objectif	2
1.1	Nouvel objectif	2
2	Vue d'ensemble des modifications	2
3	Nouveau prédicteur causal multi-horizon	3
3.1	Fichier modifié	3
3.2	Masque causal	3
3.3	Classe <code>CausalMultiHorizonPredictor</code>	4
4	Nouveau mode dans <code>JEPA.unroll</code>	4
4.1	Fichier modifié	4
4.2	Mode ajouté	4
4.3	Comportement en entraînement	4
4.4	Comportement en planning	5
5	Nouvelle loss multi-horizon	5
5.1	Fichier modifié	5
5.2	Classe ajoutée	5
5.3	Raison du choix	5
6	Branchement dans AC Video JEPA	6
6.1	Fichier modifié	6
6.2	Nouveau choix de prédicteur	6
6.3	Loss utilisée	6
7	Modifications de configuration	6
7.1	Training	6
7.2	Planning MPPI	6
8	Adaptation du planner	7
8.1	Fichier modifié	7
8.2	Détection automatique du nouveau prédicteur	7
8.3	Régularisation des actions	7
9	Tests ajoutés	7
9.1	Fichier modifié	7
9.2	Test du masque	7
9.3	Test du nouveau <code>unroll</code>	8
10	Validation effectuée	8
10.1	Limite de validation	8
11	Limites et points à surveiller	8
11.1	Historique réel en MPC	8
11.2	Encodeur Impala global	8
11.3	Comparaison expérimentale à faire	9
12	Commandes utiles	9
13	Conclusion	9

1 Contexte et objectif

Le code initial de l'exemple `examples/ac_video_jepa` utilisait un prédicteur autoregressif de type GRU. Le fonctionnement était le suivant :

$$z_t, a_t \rightarrow \hat{z}_{t+1}$$

puis :

$$\hat{z}_{t+1}, a_{t+1} \rightarrow \hat{z}_{t+2}$$

et ainsi de suite. Ce choix est simple, mais il introduit une accumulation d'erreur : une erreur sur $\hat{z}_{t+1}$ devient l'état d'entrée de la prédiction suivante.

1.1 Nouvel objectif

L'objectif demandé était de transformer le modèle vers une formulation directe multi-horizon :

$$G(z_t, s_t, a_t, \dots, a_{t+K-1}) \rightarrow (\hat{z}_{t+1}, \dots, \hat{z}_{t+K})$$

avec $K = 5$ dans la configuration par défaut.

La dépendance voulue est :

Sortie	Dépendances autorisées
$\hat{z}_{t+1}$	z_t, s_t, a_t
$\hat{z}_{t+2}$	z_t, s_t, a_t, a_{t+1}
$\hat{z}_{t+3}$	$z_t, s_t, a_t, a_{t+1}, a_{t+2}$
$\hat{z}_{t+4}$	$z_t, s_t, a_t, a_{t+1}, a_{t+2}, a_{t+3}$
$\hat{z}_{t+5}$	$z_t, s_t, a_t, a_{t+1}, a_{t+2}, a_{t+3}, a_{t+4}$

Le point essentiel est que toutes les sorties sont produites en un seul forward pass, mais chaque horizon ne voit que le préfixe d'actions qui lui correspond.

2 Vue d'ensemble des modifications

Huit fichiers ont été modifiés.

Fichier	Rôle de la modification
eb_jepa/architectures.py	Ajout du masque causal et du nouveau prédicteur Transformer multi-horizon.
eb_jepa/jepa.py	Ajout du mode <code>direct_multi_horizon</code> dans <code>JEPA.unroll</code> .
eb_jepa/losses.py	Ajout de la loss pondérée <code>MultiHorizonLoss</code> .
eb_jepa/planning.py	Adaptation du planner pour utiliser automatiquement le nouveau mode et ajout de régularisations d'actions.
examples/ac_video_jepa/main.py	Branchement du nouveau prédicteur dans l'entraînement AC Video JEPA.
examples/ac_video_jepa/cfgs/train.yaml	Configuration par défaut vers $K = 5$ et prédicteur multi-horizon.
examples/ac_video_jepa/cfgs/planning_mppi.yaml	MPC aligné sur horizon 5, coût terminal, régularisation des actions.
tests/test_jepa_output_formats.py	Ajout de tests de masque causal et de sortie multi-horizon.

3 Nouveau prédicteur causal multi-horizon

3.1 Fichier modifié

eb_jepa/architectures.py

3.2 Masque causal

Une fonction a été ajoutée :

```
build_multi_horizon_mask(
    num_context_tokens,
    num_patches,
    horizon,
    device,
)
```

Elle construit un masque d'attention booléen. Dans cette convention, **True** signifie que l'attention est bloquée.

L'ordre des tokens est :

[tokens contexte latent] + [tokens actions] + [tokens queries futures]

Le masque impose trois règles.

1. Les tokens de contexte latent ne voient que le contexte latent.
2. Le token d'action a_h voit le contexte et le préfixe $a_0, \dots, a_h$.
3. La query q_h , qui prédit $\hat{z}_{t+h+1}$, voit le contexte et $a_0, \dots, a_h$, mais pas les actions futures.

Pourquoi ce masque est nécessaire

Il ne suffit pas de bloquer directement l'accès de q_1 à a_4 . Il faut aussi empêcher les tokens intermédiaires de transporter indirectement l'information de a_4 . C'est pour cela que les tokens d'action eux-mêmes sont causaux.

3.3 Classe CausalMultiHorizonPredictor

Une nouvelle classe a été ajoutée :

```
class CausalMultiHorizonPredictor(nn.Module):
    ...
```

Son interface est :

```
state: [B, D, context, H, W]
action: [B, A, K]
return: [B, D, K, H, W]
```

Dans le cas actuel de l'encodeur Impala, la représentation est globale :

$$z_t \in \mathbb{R}^{512 \times 1 \times 1}$$

Donc le prédicteur travaille principalement avec :

$$[B, 512, context, 1, 1] + [B, 2, K] \rightarrow [B, 512, K, 1, 1]$$

Mais l'implémentation supporte aussi plusieurs patches latents si un futur encodeur produit une carte spatiale $H \times W$.

4 Nouveau mode dans JEPA.unroll

4.1 Fichier modifié

eb_jepa/jepa.py

4.2 Mode ajouté

Un nouveau mode a été ajouté :

```
unroll_mode="direct_multi_horizon"
```

Ce mode conserve les anciens modes :

- `parallel`, utilisé par l'exemple Video JEPA non actionné ;
- `autoregressive`, utilisé par l'ancien AC Video JEPA GRU ;
- `direct_multi_horizon`, ajouté pour le nouvel objectif.

4.3 Comportement en entraînement

Pendant l'entraînement, le code encode d'abord toute la séquence :

$$z_{0:T} = E(x_{0:T})$$

Puis il crée des fenêtres glissantes :

$$z_{t-c+1:t}, \quad a_t, \dots, a_{t+K-1} \rightarrow z_{t+1}, \dots, z_{t+K}$$

où c est `context_length`.

Avec la configuration actuelle :

$$c = 1, \quad K = 5$$

ce qui donne :

$$z_t, \quad a_t, \dots, a_{t+4} \rightarrow z_{t+1}, \dots, z_{t+5}$$

L'intérêt est que chaque horizon est appris directement depuis le vrai latent courant, et non depuis une prédiction précédente.

4.4 Comportement en planning

En planning, le mode reçoit :

$$z_t, \quad a_t, \dots, a_{t+K-1}$$

et produit :

$$\hat{z}_{t+1:t+K}$$

Si le planner demande un horizon plus long que celui du prédicteur, le code découpe en blocs. Pour la configuration par défaut, le planning utilise exactement $K = 5$.

5 Nouvelle loss multi-horizon

5.1 Fichier modifié

`eb_jepa/losses.py`

5.2 Classe ajoutée

```
class MultiHorizonLoss(nn.Module):
    ...
```

Elle reçoit :

```
pred: [B, D, K, H, W]
target: [B, D, K, H, W]
```

Elle applique une loss par horizon, puis pondère les horizons avec des poids croissants vers le terminal.

Par défaut :

$$\gamma = 0.5$$

Pour $K = 5$, les poids sont équivalents à :

$$[0.0625, 0.125, 0.25, 0.5, 1.0]$$

5.3 Raison du choix

Le terminal est le plus important pour le MPC, car le coût principal compare souvent $\hat{z}_{t+K}$ au but. Les horizons intermédiaires restent utiles comme supervision auxiliaire : ils forcent le modèle à apprendre une trajectoire latente plausible et pas seulement une association directe entre une séquence d'actions et un état final.

6 Branchement dans AC Video JEPA

6.1 Fichier modifié

`examples/ac_video_jepa/main.py`

6.2 Nouveau choix de prédicteur

Le script d'entraînement lit maintenant :

```
predictor_type = cfg.model.get("predictor_type", "direct_multi_horizon")
```

Deux chemins existent :

- `predictor_type: rnn` conserve l'ancien `RNNPredictor`.
- `predictor_type: direct_multi_horizon` active le nouveau prédicteur.

Cela permet de comparer l'ancien modèle et le nouveau sans supprimer le comportement précédent.

6.3 Loss utilisée

Si le nouveau mode est actif, le training utilise :

```
MultiHorizonLoss(...)
```

Sinon, il conserve :

```
SquareLossSeq()
```

7 Modifications de configuration

7.1 Training

Fichier :

`examples/ac_video_jepa/cfgs/train.yaml`

Les changements principaux sont :

```
nsteps: 5
predictor_type: direct_multi_horizon
context_length: 1
pred_dim: 512
predictor_depth: 4
predictor_heads: 8
horizon_loss_gamma: 0.5
horizon_loss_type: smooth_l1
```

La valeur $K = 5$ correspond directement à la formulation discutée.

7.2 Planning MPPI

Fichier :

`examples/ac_video_jepa/cfgs/planning_mppi.yaml`

Les changements principaux sont :

```
plan_length: 5
sum_all_diffs: false
action_l2_coeff: 0.01
action_smoothness_coeff: 0.05
```

Le coût MPC devient principalement terminal :

$$J = d(\hat{z}_{t+5}, z_{\text{goal}})$$

avec régularisation légère des actions.

8 Adaptation du planner

8.1 Fichier modifié

eb_jepa/planning.py

8.2 Détection automatique du nouveau prédicteur

Le planner détecte :

```
predictor.direct_multi_horizon
```

Si ce flag existe, il appelle :

```
unroll_mode="direct_multi_horizon"
```

Sinon, il conserve l'ancien mode :

```
unroll_mode="autoregressive"
```

8.3 Régularisation des actions

Deux coefficients optionnels ont été ajoutés :

```
action_l2_coeff
action_smoothness_coeff
```

Le coût devient :

$$J = d(\hat{z}_{t+K}, z_g) + \lambda \frac{1}{K} \sum_i \|a_i\|^2 + \mu \frac{1}{K-1} \sum_i \|a_i - a_{i-1}\|^2$$

Cela stabilise le planning et évite des séquences d'actions inutilement violentes.

9 Tests ajoutés

9.1 Fichier modifié

tests/test_jepa_output_formats.py

9.2 Test du masque

Test ajouté :

```
test_direct_multi_horizon_mask()
```

Il vérifie que :

- les actions ne voient pas les actions futures ;
- les queries ne voient pas les actions futures ;
- les queries ne communiquent pas entre elles, sauf auto-attention à elles-mêmes.

9.3 Test du nouveau unroll

Test ajouté :

```
test_direct_multi_horizon_unroll_with_loss()
```

Il vérifie :

- la forme de sortie en entraînement ;
- la présence d’une loss finie ;
- la forme de sortie en planning sans loss.

10 Validation effectuée

Les validations suivantes ont été exécutées avec succès :

```
python3 -m py_compile ...
git diff --check
```

Cela garantit :

- absence d’erreurs syntaxiques Python ;
- absence de problèmes de whitespace dans le diff.

10.1 Limite de validation

Les tests PyTorch complets n’ont pas pu être lancés dans l’environnement courant, car :

- `torch` n’est pas installé dans le `python3` disponible ;
- `pytest` n’est pas disponible ;
- `uv` n’est pas disponible ;
- le `python3` système est en version 3.9.6, alors que le projet demande Python 3.12.

11 Limites et points à surveiller

11.1 Historique réel en MPC

Le code supporte `context_length > 1` au niveau du prédicteur et du training. Cependant, le planner actuel reçoit principalement l’observation courante.

Si l’objectif final est vraiment :

$$G(z_{t-c+1:t}, a_t, \dots, a_{t+K-1})$$

alors il faudra ajouter une mémoire d’observations dans `GCagent`, afin que le MPC fournisse les derniers états réels au modèle au lieu d’un contexte paddé.

11.2 Encodeur Impala global

L’encodeur actuel produit une représentation globale :

$$[B, 512, T, 1, 1]$$

Donc le mécanisme query par patch fonctionne, mais avec un seul patch. Si l’on veut se rapprocher davantage de V-JEPA 2, il faudra probablement utiliser un encodeur qui conserve une grille de tokens spatiaux.

11.3 Comparaison expérimentale à faire

Il faudra comparer :

- ancien `RNNPredictor` autoregressif;
- nouveau prédicteur direct multi-horizon;
- `context_length = 1` contre `context_length > 1`;
- coût MPC terminal seul contre coût sur tous les horizons.

12 Commandes utiles

Pour lancer l'entraînement avec le nouveau mode :

```
python -m examples.ac_video_jepa.main \  
--fname examples/ac_video_jepa/cfgs/train.yaml
```

Pour revenir à l'ancien prédicteur GRU :

```
python -m examples.ac_video_jepa.main \  
--fname examples/ac_video_jepa/cfgs/train.yaml \  
model.predictor_type=rnn
```

Pour tester l'idée avec un contexte plus long :

```
python -m examples.ac_video_jepa.main \  
--fname examples/ac_video_jepa/cfgs/train.yaml \  
model.context_length=3
```

13 Conclusion

La modification principale est conceptuelle : le modèle n'est plus obligé de propager ses propres erreurs horizon après horizon. Il apprend maintenant une fonction directe :

$$G(z_t, a_t, \dots, a_{t+4}) \rightarrow (\hat{z}_{t+1}, \dots, \hat{z}_{t+5})$$

avec une causalité stricte :

$$\hat{z}_{t+h} \not\leftarrow a_{t+h}, a_{t+h+1}, \dots, a_{t+4}$$

Ce changement aligne mieux le modèle avec le MPC : le planner peut scorer directement une séquence candidate d'actions via l'état latent terminal prédit, puis exécuter seulement la première action et replanifier.